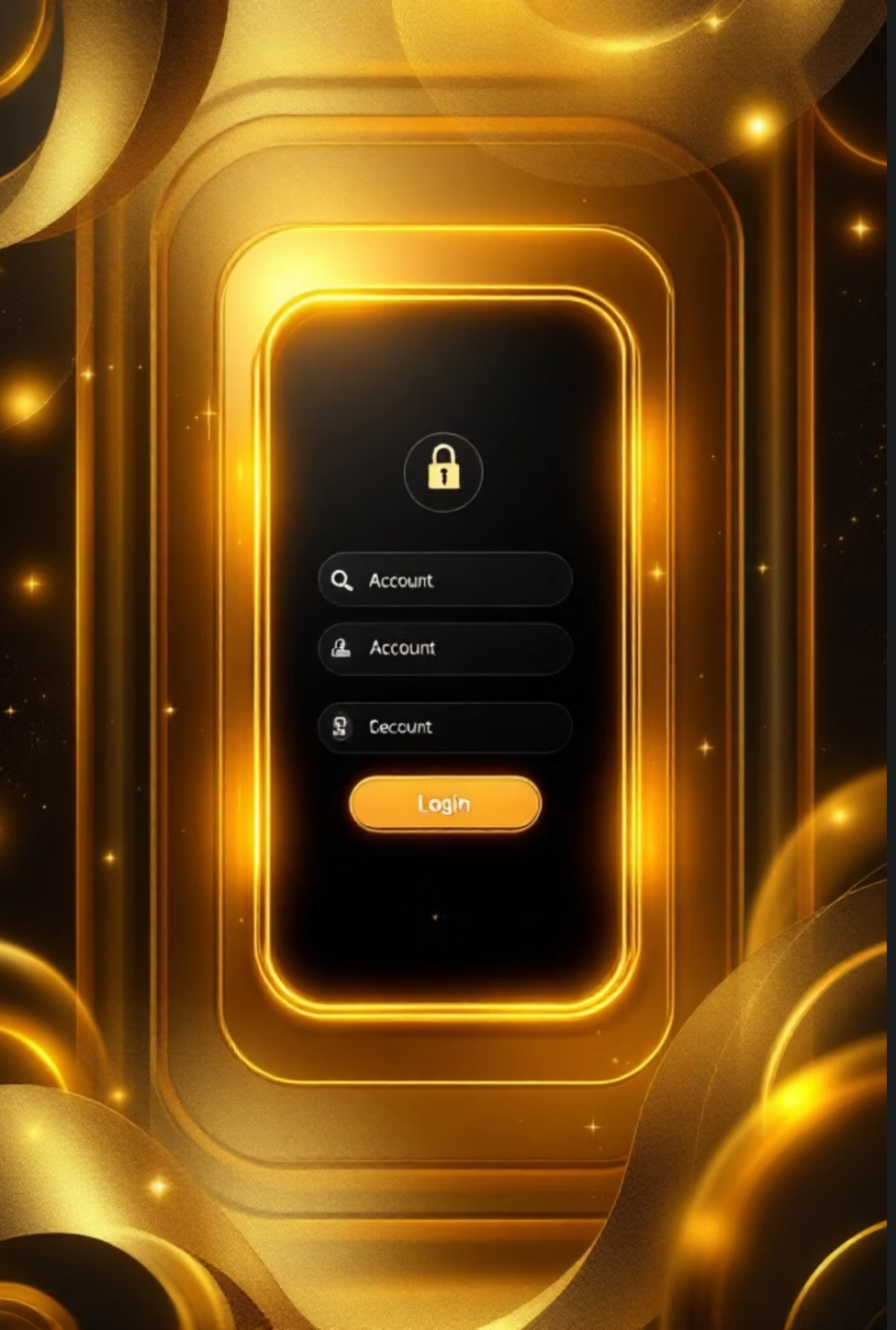


Bank Account Simulation System in C

A Modular Approach to Secure Digital Banking and Fund Management

Presented by: [Dhruv jadhav(1262252800)],
[Ashray shingala(1262251420)],
[Aryan Kulkarni(1262252488)]
Division: [10]

Course: Fundamentals of Programming (FOP)

System Overview: Foundation and Features

Core C Implementation

Built entirely in C, ensuring high performance, direct memory management, and compatibility with various operating systems.

Secure Data Handling

Utilises file I/O for persistent storage of account data, safeguarding against system reboots and ensuring data integrity.

Dual Menu System

Separates functionalities for Administrator and Customer, each with distinct access controls and operations.

Comprehensive Transactions

Supports essential banking operations like deposits, withdrawals, transfers, and interest applications for savings accounts.

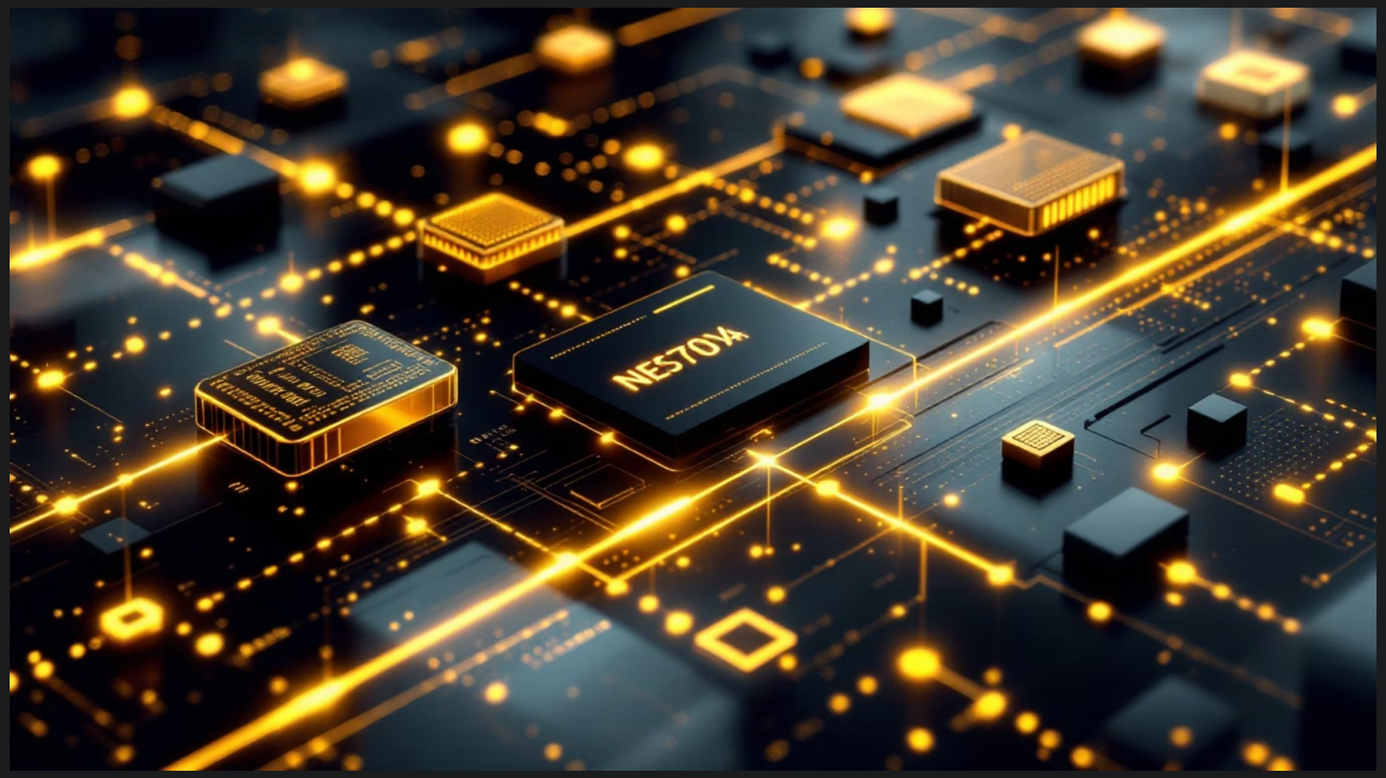
FOUNDATIONS

Data Architecture: The Core Structure

The foundation of our simulation lies in a meticulously defined data architecture, designed for efficiency and reliability.

Structure Definition (struct Account)

- accNum **(int):** The unique primary key for identifying each user.
- name **(char[50]):** An array to securely store account holder details.
- balance **(float):** Utilising floating-point precision for accurate currency representation.
- isSavings **(int):** A boolean-style flag distinguishing between Savings and Current account types.



Memory Management

All critical data is stored within the **Data Segment** of the program memory, ensuring high-speed access and optimal performance during the demonstration.

Data Structures: Account Management

The backbone of our system is the struct Account, meticulously designed to hold all necessary account details. This structure ensures comprehensive data management for each customer.

```
struct Account {
    int accNum;
    int pin;
    char name[50];
    char mobile[15];
    char email[50];
    float balance;
    int isSavings;
    float todayWithdrawn;
    char
history[MAX_TRANSACTIONS][100];
    int transCount;
};
```

OPERATIONAL FLOW

System Logic & Operational Flow

Our system is structured around a clear, two-menu architecture to streamline user interaction and manage access control effectively.

Admin Menu

Dedicated to account creation, user management, and overall system maintenance tasks.

Customer Menu

Protected by a robust login gate, requiring PIN-based authentication for access to personal banking functionalities.

Validation Logic

- Strictly prevents the creation of duplicate account IDs, maintaining data integrity.
- Thoroughly validates account balances to ensure sufficient funds are available before any withdrawal or transfer can proceed.
- Ensures that **Interest Calculation** functionality is exclusively applied to Savings accounts.

Core Functionalities: Admin Dashboard

The Admin Dashboard provides powerful tools for bank management, accessible only with a secure password. Administrators can oversee and manage all aspects of the banking system.

1

ADMINISTRATION

Account Creation

Generate unique account numbers and set initial details for new customers.

2

VISIBILITY

View All Accounts

Access a comprehensive list of all customer accounts with their current balances.

3

SEARCH & LOCATE

Search Account

Efficiently locate specific accounts using account numbers for quick access to details.

4

MAINTENANCE

Delete Account

Securely remove accounts from the system after OTP verification.

5

FINANCIAL OVERSIGHT

Total Bank Balance

Monitor the aggregated balance across all active bank accounts for financial reporting.

HIGHLIGHTS

Innovative Features & Capabilities

Our system incorporates several cutting-edge features designed to enhance user experience and functionality.

Dynamic Account Upgrading

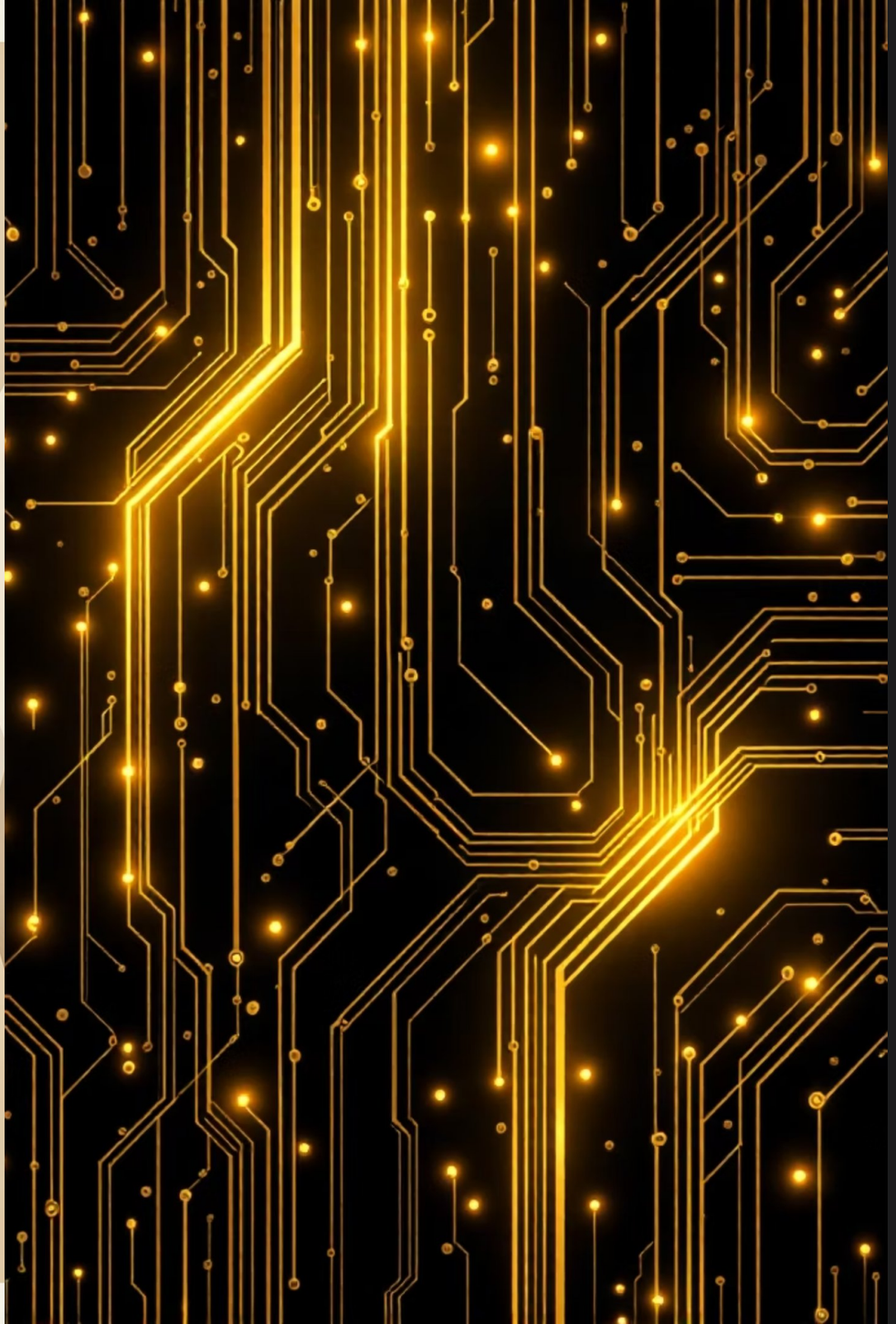
Facilitates seamless upgrades for "Current" account users to "Savings" accounts, preserving their existing ID and balance.

Fund Transfer Engine

A sophisticated, multi-step algorithm meticulously updates two distinct structure indices simultaneously for secure transfers.

Transaction Logging

Utilises sprintf() to dynamically generate a live **Mini Statement**, providing an immediate record of the last performed activity.



Customer Operations: Secure and User-Friendly

Customers interact with the system through a secure login, accessing a suite of personal banking services. Each transaction is designed for ease of use while maintaining high security standards.

Deposit Funds

Add money to an account with instant balance updates.

Withdraw Funds

Safely remove money, adhering to daily withdrawal limits.

Transfer Funds

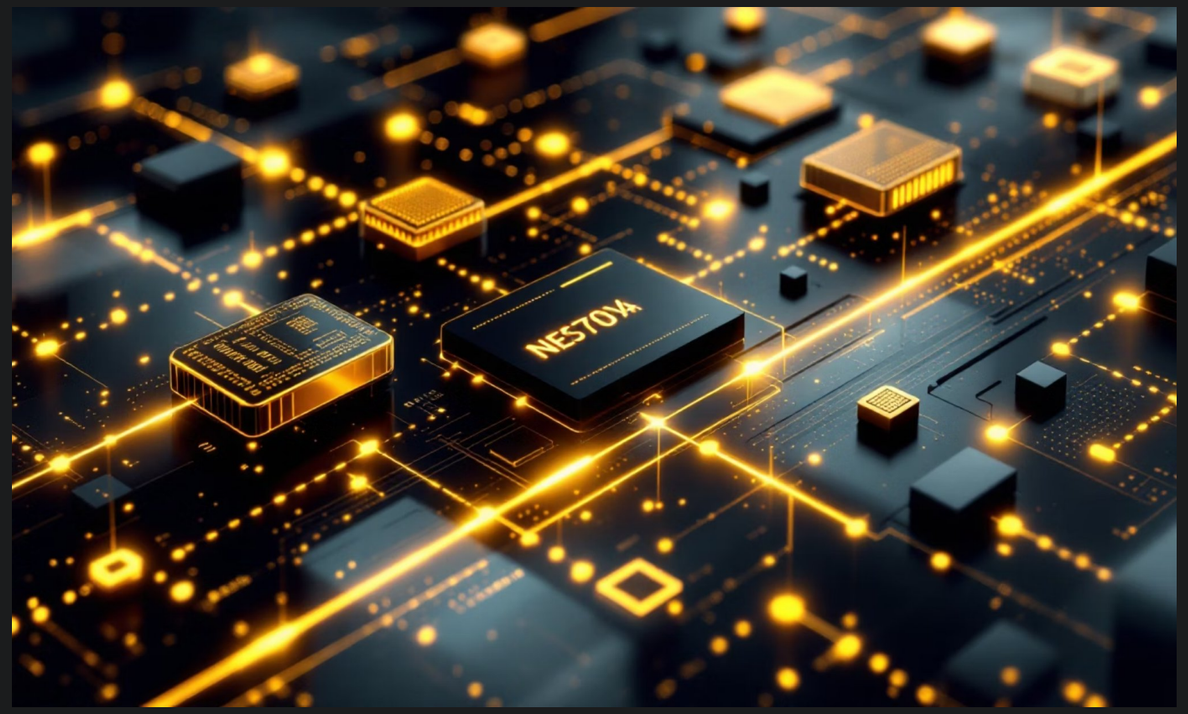
Move money between accounts securely, requiring recipient account details.

Apply Interest

Automated interest calculation for savings accounts, enhancing customer growth.

Security & Verification: Protecting Your Assets

Security is paramount. Our system employs multi-layered authentication and verification processes to protect sensitive account information and transactions.



Authentication

PIN-based login for all customer and administrator access.

OTP Verification

One-Time Password (OTP) verification for critical operations like account creation and deletion, sent to registered mobile and email.

Forgot PIN

Secure PIN reset mechanism using OTP verification.

Daily Withdrawal Limit

Automated tracking to prevent exceeding DAILY_WITHDRAW_LIMIT for enhanced security.

Transaction History & Balance Enquiry

Transparency and instant information are key to customer satisfaction. Our system provides easy access to recent transactions and current balance details.

Mini Transactions:

Allows customers to view their most recent 5 transactions, offering a quick overview of account activity. Each transaction is recorded with a clear description, providing a concise audit trail.

```
sprintf(msg,"Deposited %.2f",amt);
addHistory(&bank[i],msg);
```

The addHistory function diligently logs all significant account events.

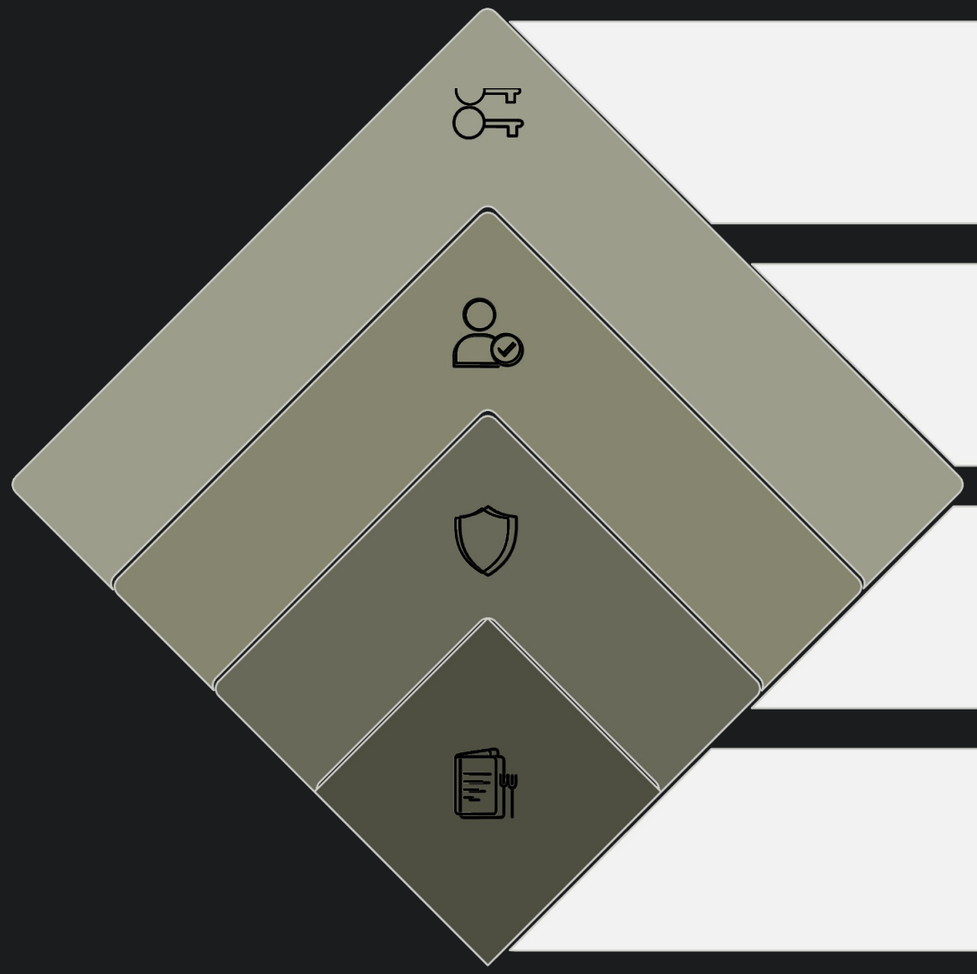
Check Balance:

Provides an immediate display of the customer's name, account number, and current available balance. This feature ensures customers are always informed about their financial standing.

```
printf("Name: %s\n",bank[i].name);
printf("Account No: %d\n",bank[i].accNum);
printf("Balance: %.2f\n",bank[i].balance);
```

Ensuring accuracy and real-time updates for customer convenience.

System Architecture: Control Flow



Forgot PIN

Account, OTP, reset PIN

Customer Login

Account and PIN authentication

Admin Login

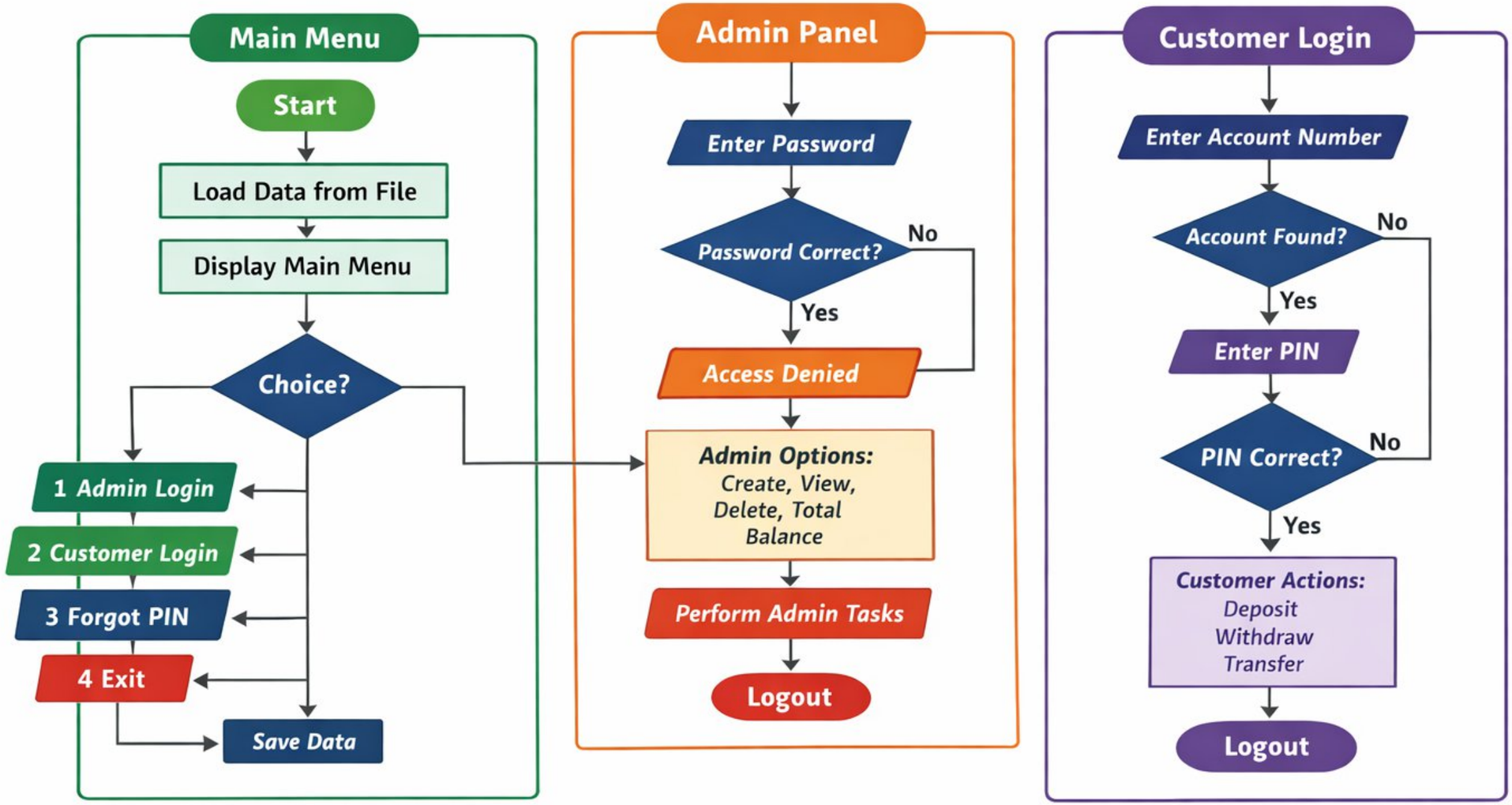
Password check then dashboard

Main Menu

Admin, Customer, PIN help

The robust control flow ensures a seamless user experience for both administrators and customers, guiding them through the various functionalities with clear pathways.

BANK ACCOUNT SYSTEM



For your attention and engagement.

- **Attention**

In today's digital world, managing money efficiently is more important than ever. From students receiving scholarships to businesses handling daily transactions, bank accounts play a crucial role in financial management. However, many people do not fully understand how a banking system works behind the scenes.
The **Bank Account System** project captures attention by demonstrating how technology simplifies financial operations like deposits, withdrawals, balance inquiries, and transaction records. It shows how a simple program can simulate real-world banking operations in a secure and organized manner.

- **Engagement**

This project actively engages users by allowing them to:
Create new bank accounts
Deposit and withdraw money
Check account balance
View transaction history
By interacting with the system, users gain practical knowledge about how banking software functions. It enhances understanding of concepts like data storage, validation, authentication, and financial accuracy.
The project also encourages logical thinking and problem-solving, as users explore how transactions are processed and recorded securely.

Questions & Discussion

We welcome your questions and insights into our Bank Account Simulation System.

- How might we scale this system for a larger user base?
- What additional security measures could be integrated?
- What are the potential applications for this system beyond a simulation?

Contact Information

Email

teammitwpu@gmail.com

Website

www.projectsim.com

GitHub

github.com/bank-sim

We are available for further inquiries and collaboration.